SAMSUNG

LIFE Makers EGYPT

E-Commerce Pipeline System

End-to-End Data Engineering & Analytics Project

Samsung Innovation Campus & Life Maker

Team Members

Abdullah Emad Abdulaziz

Youssef Mahmoud Abo Ali

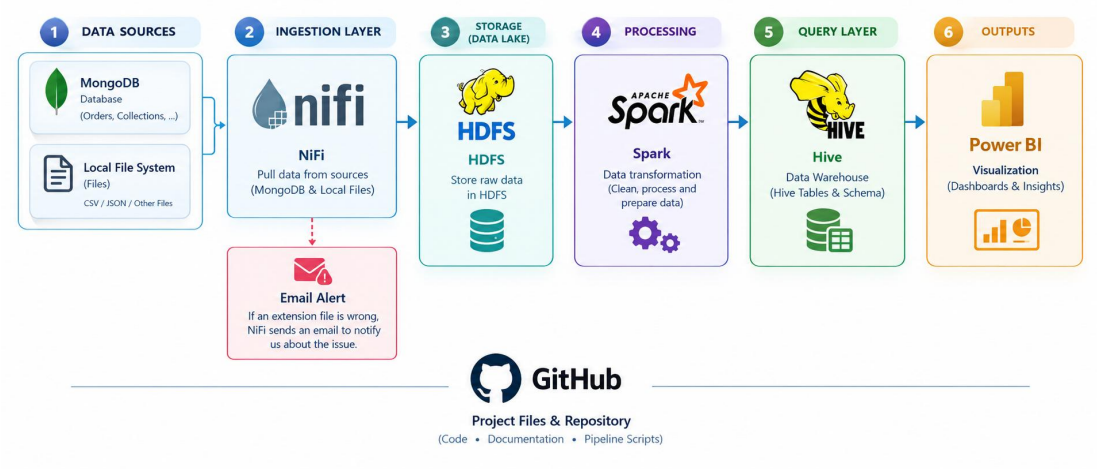
Hamza Mohamed Ali

Omar Fathy Mohamed

E-commerce Data Pipeline

Project: Orders & Payments:

➢ Data Pipeline Flow:



- **Data Ingestion:** Uses **Apache NiFi** to extract and route data from different sources.
- **Data Storage:** Stores the raw, unaltered data in **HDFS** as the Data Lake layer.
- **Data Transformation:** Uses **Apache Spark / PySpark** to clean, process, and transform the raw data.
- **Data Warehouse:** Uses **Apache Hive** to create structured tables and a Data Warehouse based on a **Star Schema**.
- **Data Quality & Monitoring:** NiFi validates incoming files and sends an **Email Alert** when an invalid file extension or pipeline issue is detected.
- **Business Intelligence:** Connects the processed data to **Power BI** to build dashboards, KPIs, and business insights.
- **Version Control:** Stores the project's code, pipeline configurations, documentation, and scripts in **GitHub**.

➢ Data Exploration:

We begin to explore the **“orders”** collection in the **“e-commerce”** database in MongoDB to know the attributes inside it:

```
test> use e-commerce;
switched to db e-commerce
e-commerce> show collections
orders
e-commerce> db.orders.find()
```

A sample from the data:

```
[
  {
    _id: ObjectId('6a997e76a90fcfc28ca6b1ff'),
    order_id: 643,
    customer_id: 426,
    order_date: '2023-11-05',
    total_value: 517.12,
    payment_status: 'refunded',
    items: [
      { product_id: 41, quantity: 6 },
      { product_id: 52, quantity: 2 },
      { product_id: 41, quantity: 8 }
    ]
  },
  {
    _id: ObjectId('6a997e76a90fcfc28ca6b200'),
    order_id: 334,
    customer_id: 588,
    order_date: '2024-11-13',
    total_value: 316.68,
    payment_status: 'pending',
    items: [ { product_id: 57, quantity: 6 }, { product_id: 15, quantity: 8 } ]
  },
```

- **Note:** We notice that **“items”** attribute has **“nested arrays”** which contain the order items and their quantities in each order.

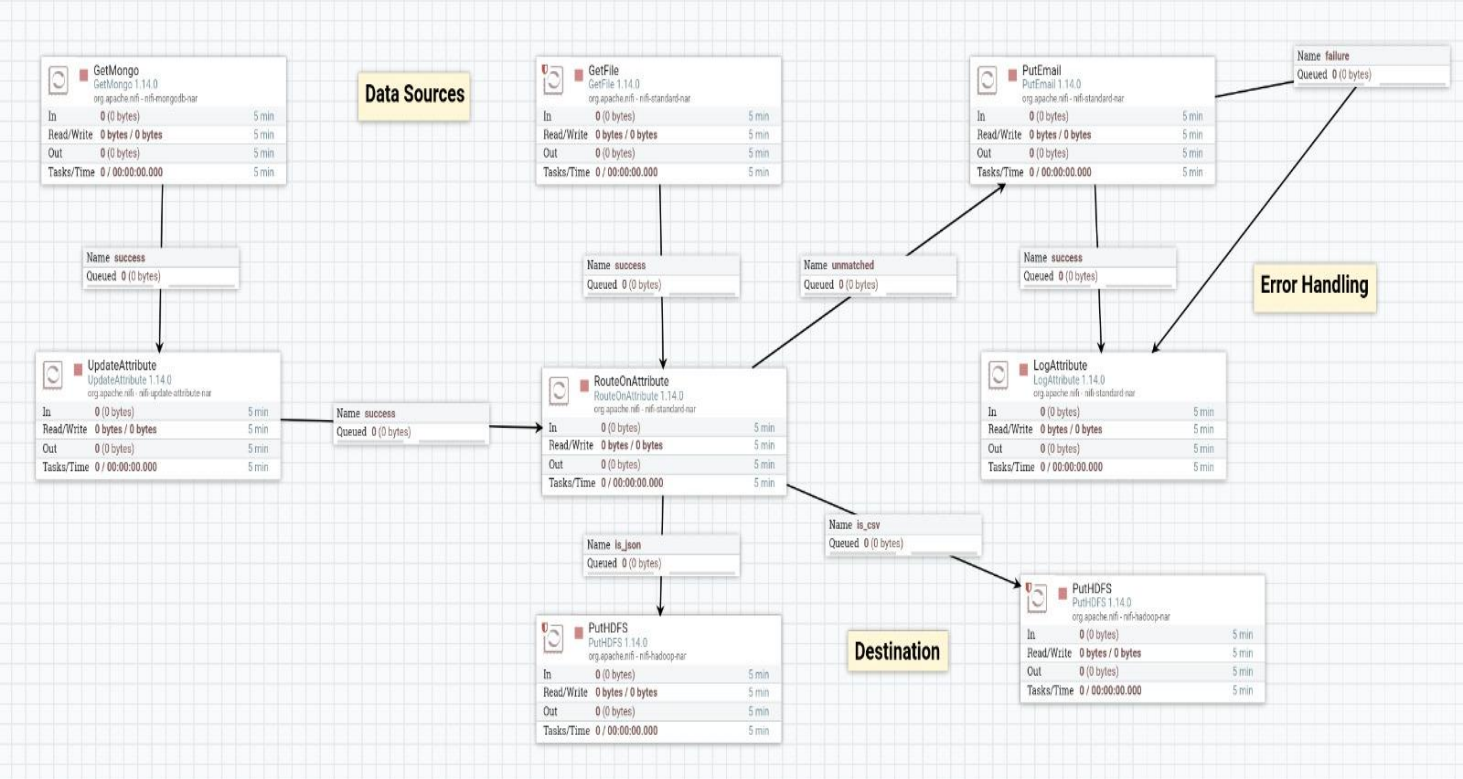
➢ Prepare the structure of directories in Data Lake (HDFS):

```
[student@localhost e-commerce]$ hdfs dfs -mkdir /user/student/e-commerce
[student@localhost e-commerce]$ hdfs dfs -mkdir /user/student/e-commerce/raw_data
[student@localhost e-commerce]$ hdfs dfs -mkdir /user/student/e-commerce/cleaned_data
[student@localhost e-commerce]$ hdfs dfs -ls /user/student/e-commerce
Found 2 items
drwxr-xr-x   - student student          0 2026-09-03 09:08 /user/student/e-commerce/cleaned_data
drwxr-xr-x   - student student          0 2026-09-03 09:06 /user/student/e-commerce/raw_data
[student@localhost e-commerce]$ hdfs dfs -mkdir /user/student/e-commerce/raw_data/csv_files
[student@localhost e-commerce]$ hdfs dfs -mkdir /user/student/e-commerce/raw_data/json_files
[student@localhost e-commerce]$ hdfs dfs -ls /user/student/e-commerce/raw_data
Found 2 items
drwxr-xr-x   - student student          0 2026-09-03 09:10 /user/student/e-commerce/raw_data/csv_files
drwxr-xr-x   - student student          0 2026-09-03 09:10 /user/student/e-commerce/raw_data/json_files
[student@localhost e-commerce]$
```

➢ Data Ingestion & Raw Zone Storage:

We use Apache NiFi as an ingestion tool to get files and store them in Raw Zone (HDFS):

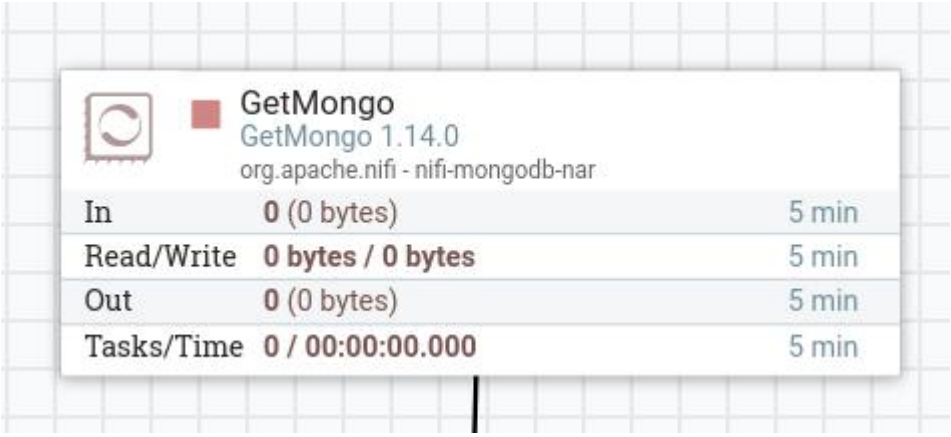
- **NiFi Template:**



- **Processors:**

1. GetMongo:

To get orders JSON file from MongoDB.



- Its configuration:

Property	Value
Client Service	MongoDBControllerService
Mongo URI	No value set
Mongo Database Name	**e-commerce**
Mongo Collection Name	**orders**

Client Service: **MongoDBControllerservice:**

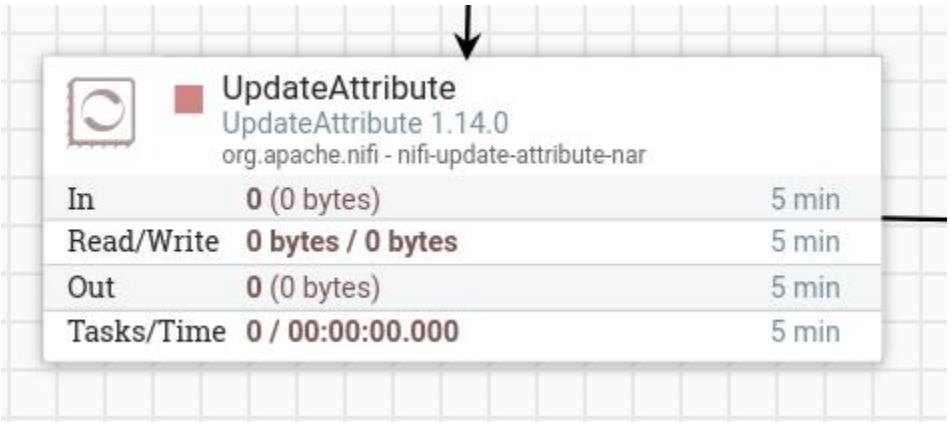
Provides the connection between MongoDB and NiFi.

Property	Value
Mongo URI	**mongodb://127.0.0.1:27017**

Mongo URL: Where MongoDB is running.

2. UpdateAttribute:

We face a problem that **GetMongo** processor remove the extension (.json) from the name of the FlowFile so we use **UpdateAttribute** to add the extension to the name of the file again.

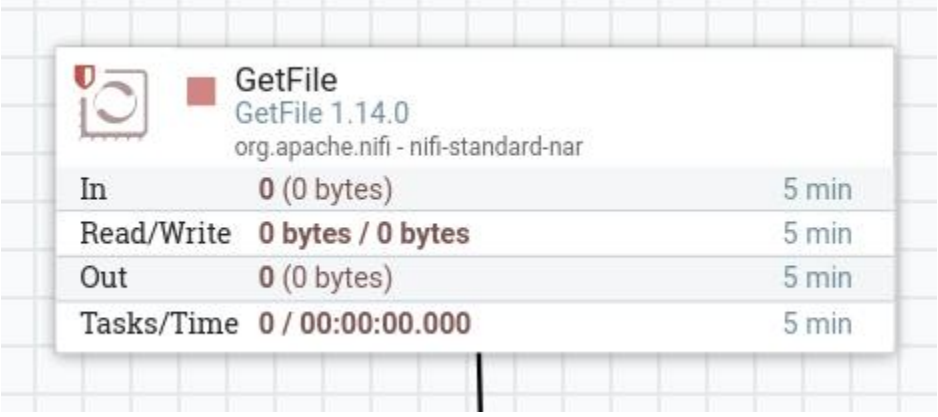


- Its configuration:

We add a property named **"filename"** that add the extension (.json) to the file name again if its JSON file.

3. GetFile:

To get the CSV files from the local file system.



- Its configuration:

Property	Value
Input Directory	/home/student/e-commerce
File Filter	[^\.].*

Input Directory:

The path for the directory in the local file system.

4. RouteOnAttribute:

To route FlowFiles to different paths based on conditions.

- Its configuration:

Property	Value	
Routing Strategy	Route to Property name	
is_csv	${filename:endsWith('.csv')}	
is_json	${filename:endsWith('.json')}	

is_csv:

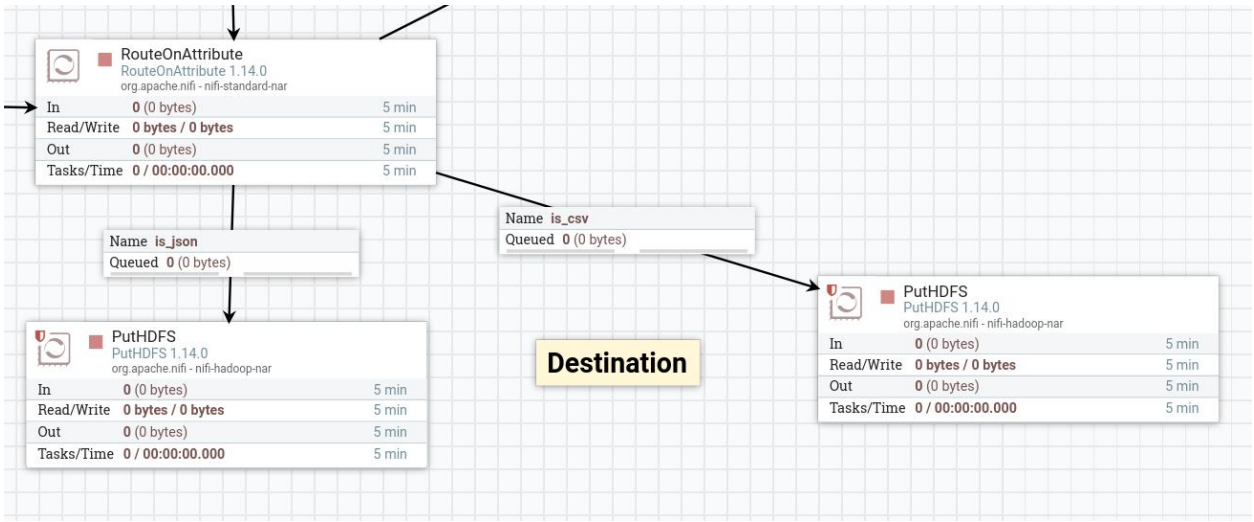
A property that checks if the file type is **CSV**.

is_json:

A property that checks if the file type is **JSON**.

5. PutHDFS:

To save FlowFiles in the HDFS.



- Its configuration:

Directory	/user/student/e-commerce/raw_data/json_files

The storage path for the JSON files.

Directory	/user/student/e-commerce/raw_data/csv_files

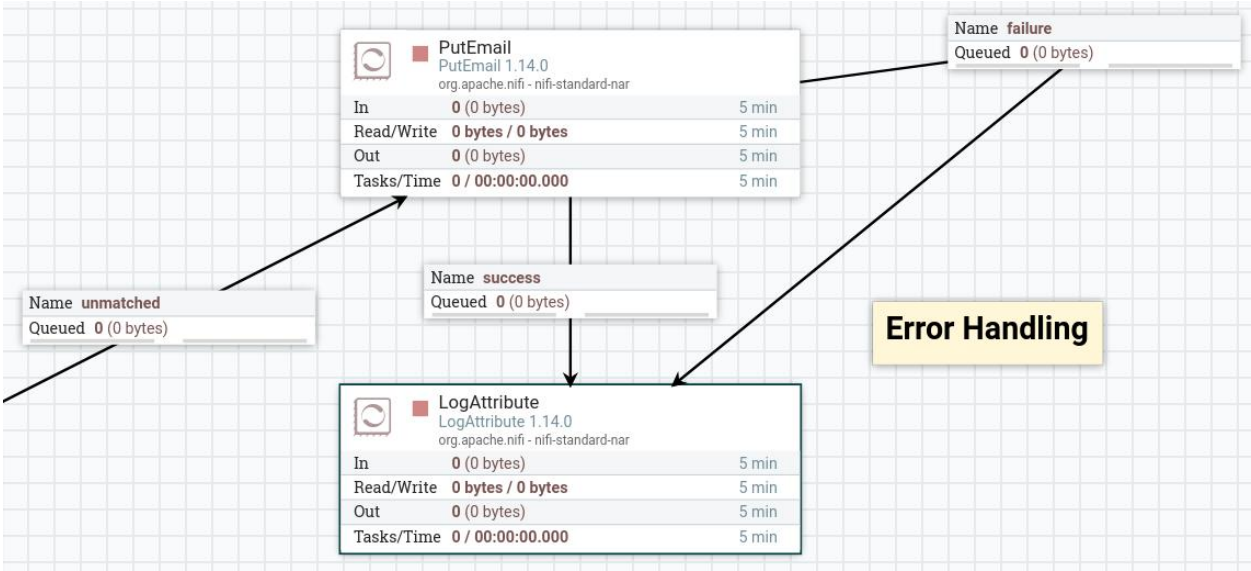
The storage path for the CSV files.

- The output after running the processor:

```
[student@localhost ~]$ hdfs dfs -ls /user/student/e-commerce/raw_data/csv_files
Found 2 items
-rw-r--r--   1 root student      41315 2026-09-04 12:36 /user/student/e-commerce/raw_data/csv_files/customers.csv
-rw-r--r--   1 root student       2549 2026-09-04 12:35 /user/student/e-commerce/raw_data/csv_files/products.csv
[student@localhost ~]$ hdfs dfs -ls /user/student/e-commerce/raw_data/json_files
Found 1 items
-rw-r--r--   1 root student     372945 2026-09-04 12:35 /user/student/e-commerce/raw_data/json_files/3e878ee9-a0d6-407e-9d20-8f725311ac5
9.json
[student@localhost ~]$
```

6. PutEmail:

Send an email from a NiFi Data Flow.



If the file type checked by the **RouteOnAttribute** processor is neither JSON nor CSV, the file will enter the **unmatched** relationship and then be sent to the **PutEmail** processor, which sends an alert to a specified email account.

- Its configuration:

From	**hamza.data.engineer777@gmail.com**
To	engineerdata347@gmail.com
CC	No value set
BCC	No value set
Subject	**Alert of unmatched type file from NiFi**
Message	File (${filename}) did not match expected extensi...

From: The **Sender**.

To: The **Recipient**.

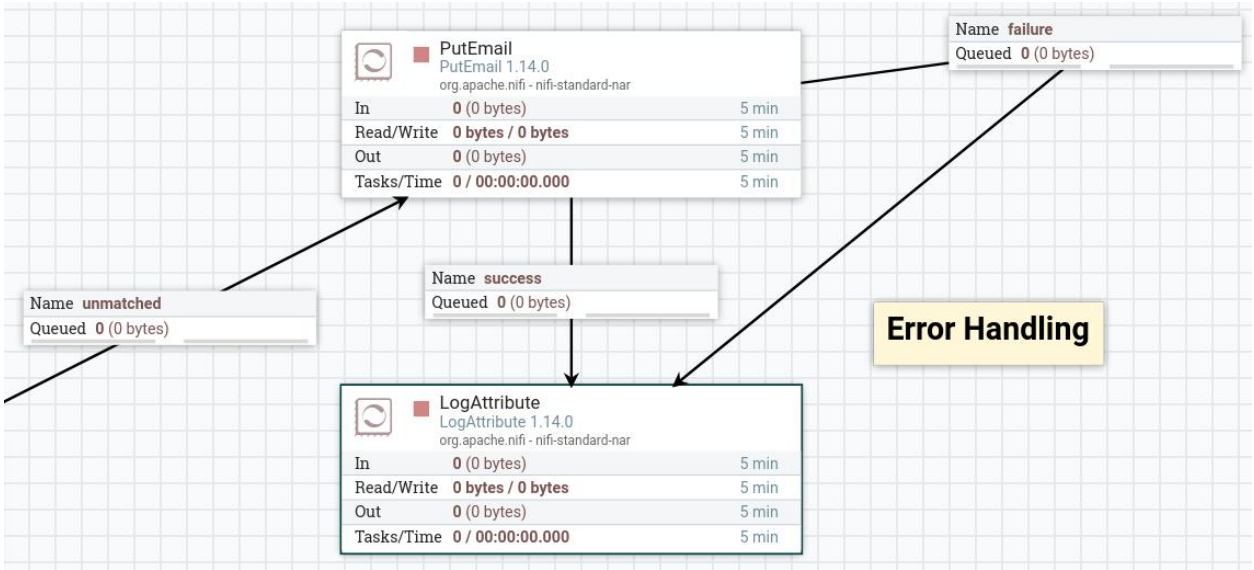
Subject: **Email Subject**.

Message: The **Body** of the **Email**.

- The output of the **PutEmail** if the file type unmatched:

7. LogAttribute:

Used for debugging and monitoring a NiFi Data Flow.



We use **LogAttribute** processor to check the success or failure of the **PutEmail** processor.

➢ Data Transformation:

We use Apache Spark as a transformation tool:

- **PySpark Script:**

- Import **PySpark** libraries:

```
from pyspark.sql import *
from pyspark.sql.functions import *
from pyspark.sql.window import Window
```

- Read the orders JSON file from its path:

```
orders = spark.read.option("multiLine", True) \
    .json("/user/student/e-commerce/raw_data/json_files/*.json")
```

- Print the Schema of the orders:

```
orders.printSchema()
```

```
root
 |-- _id: struct (nullable = true)
 |    |-- $oid: string (nullable = true)
 |-- customer_id: long (nullable = true)
 |-- items: array (nullable = true)
 |    |-- element: struct (containsNull = true)
 |    |    |-- product_id: long (nullable = true)
 |    |    |-- quantity: long (nullable = true)
 |-- order_date: string (nullable = true)
 |-- order_id: long (nullable = true)
 |-- payment_status: string (nullable = true)
 |-- total_value: double (nullable = true)
```

- Drop the **"_id"** column that is created by default in **MongoDB**:

```
orders = orders.drop("_id")
```

- Show the shape of orders table:

```
orders.show(5)
```

```
+-----------+--------------------+----------+--------+--------------+-----------+
|customer_id|               items|order_date|order_id|payment_status|total_value|
+-----------+--------------------+----------+--------+--------------+-----------+
|        426|[{41, 6}, {52, 2}...|2023-11-05|     643|      refunded|     517.12|
|        588|  [{57, 6}, {15, 8}]|2024-11-13|     334|       pending|     316.68|
|         59|[{2, 3}, {14, 5},...|2023-09-08|     611|      refunded|     879.28|
|        509|[{55, 3}, {46, 4}...|2023-01-31|      67|        failed|       null|
|        427|    [{2, 7}, {7, 5}]|2023-05-04|     224|     completed|     258.89|
+-----------+--------------------+----------+--------+--------------+-----------+
only showing top 5 rows
```

Note: **Items** column contains **nested arrays** that contain order items and their quantities.

- Check and drop duplicate values:

```
orders.count()
```

```
1000
```

```
orders = orders.dropDuplicates()
```

```
orders.count()
```

```
920
```

- Separate each order item and its quantity in a single record using **explode** function:

```
orders_exploded = orders.select(
    col("order_id"),
    col("customer_id"),
    col("order_date"),
    col("payment_status"),
    explode(col("items")).alias("item_detail")
)
```

```
orders_exploded.show(5)
```

```
+--------+-----------+----------+--------------+-----------+
|order_id|customer_id|order_date|payment_status|item_detail|
+--------+-----------+----------+--------------+-----------+
|     459|        465|2024-01-29|     completed|   {55, 10}|
|     459|        465|2024-01-29|     completed|    {13, 6}|
|     459|        465|2024-01-29|     completed|     {4, 8}|
|     459|        465|2024-01-29|     completed|    {67, 6}|
|     866|        333|2024-04-14|     completed|    {21, 4}|
+--------+-----------+----------+--------------+-----------+
only showing top 5 rows
```

Note: Each record in **item_detail** column contains product_id and its quantity in an array.

- Separate product_id and its quantity in two columns:

```
windowSpec = Window.partitionBy("order_id").orderBy("item_detail.product_id")

orders_details = orders_exploded.select(
    concat(col("order_id"), lit("_"), row_number().over(windowSpec)).alias("order_line_id"),
    col("order_id"),
    col("customer_id"),
    col("order_date"),
    col("item_detail.product_id").alias("product_id"),
    col("item_detail.quantity").cast("int").alias("sales_quantity"),
    col("payment_status")
)
```

```
orders_details.show(5)
```

```
[Stage 93:=======================================>             (147 + 6) / 200]
```

```
+-------------+--------+-----------+----------+----------+--------------+--------------+
|order_line_id|order_id|customer_id|order_date|product_id|sales_quantity|payment_status|
+-------------+--------+-----------+----------+----------+--------------+--------------+
|         26_1|      26|        145|2024-06-07|        18|            10|     completed|
|         26_2|      26|        145|2024-06-07|        24|             8|     completed|
|         26_3|      26|        145|2024-06-07|        50|             7|     completed|
|         26_4|      26|        145|2024-06-07|        54|            10|     completed|
|         29_1|      29|        501|2023-06-21|         4|             4|        failed|
+-------------+--------+-----------+----------+----------+--------------+--------------+
only showing top 5 rows
```

- Read the products CSV file from its path:

```
products = spark.read.option("header", "true") \
    .option("inferSchema", "true") \
    .csv("/user/student/e-commerce/raw_data/csv_files/products.csv")
```

- Print the Schema and the shape of the products table:

```
products.printSchema()
```

```
root
 |-- product_id: integer (nullable = true)
 |-- product_name: string (nullable = true)
 |-- category: string (nullable = true)
 |-- price: double (nullable = true)
```

```
products.show(5)
```

```
+----------+--------------------+-------------+------+
|product_id|        product_name|     category| price|
+----------+--------------------+-------------+------+
|         1|Non-Stick Frying Pan|Home & Garden|  86.4|
|         2|           Webcam HD|  Electronics|236.02|
|         3|           Jump Rope|       Sports| 79.23|
|         4|  Scented Candle Set|Home & Garden| 83.78|
|         5|       Puzzle 1000pc|         Toys| 35.54|
+----------+--------------------+-------------+------+
only showing top 5 rows
```

- Join orders with products and check if any product in the orders table has no price in products table:

```
orders_with_products = orders_details.join(products, "product_id", how="left")

orders_with_products.filter(col("price").isNull()).show()
```

```
+----------+-------------+--------+-----------+----------+--------------+--------------+------------+--------+----
-+
|product_id|order_line_id|order_id|customer_id|order_date|sales_quantity|payment_status|product_name|category|pric
e|
+----------+-------------+--------+-----------+----------+--------------+--------------+------------+--------+----
-+
+----------+-------------+--------+-----------+----------+--------------+--------------+------------+--------+----
-+
```

Note: All products have price.

- Create a new column that calculate the sales per product per order:

```
orders_with_products = orders_with_products.withColumn(
    "sales_per_product_per_order", col("sales_quantity") * col("price"))

orders_with_products.show(5)
```

```
[Stage 116:==================================>                    (127 + 7) / 200]
```

```
+----------+-------------+--------+-----------+----------+--------------+--------------+------------------+-------
------+------+--------------------------+
|product_id|order_line_id|order_id|customer_id|order_date|sales_quantity|payment_status|      product_name|     ca
tegory| price|sales_per_product_per_order|
+----------+-------------+--------+-----------+----------+--------------+--------------+------------------+-------
------+------+--------------------------+
|        18|         26_1|      26|        145|2024-06-07|            10|     completed|       Foam Roller|
Sports| 33.45|                     334.5|
|        24|         26_2|      26|        145|2024-06-07|             8|     completed|     Action Figure|
Toys| 33.04|                    264.32|
|        50|         26_3|      26|        145|2024-06-07|             7|     completed|      Wool Sweater|     Cl
othing|102.68|                    718.76|
|        54|         26_4|      26|        145|2024-06-07|            10|     completed|  External SSD 1TB|  Elect
ronics|159.13|                   1591.3|
|         4|         29_1|      29|        501|2023-06-21|             4|        failed|Scented Candle Set|Home &
Garden| 83.78|                    335.12|
+----------+-------------+--------+-----------+----------+--------------+--------------+------------------+-------
------+------+--------------------------+
only showing top 5 rows
```

- Select the columns that will be in the fact table:

```
orders_final = orders_with_products.select(
    "order_line_id", "order_id", "customer_id", "order_date",
    "product_id", "sales_quantity", "sales_per_product_per_order", "payment_status")
```

```
orders_final.printSchema()

orders_final.show(5, truncate=False)
```

```
root
 |-- order_line_id: string (nullable = true)
 |-- order_id: long (nullable = true)
 |-- customer_id: long (nullable = true)
 |-- order_date: string (nullable = true)
 |-- product_id: long (nullable = true)
 |-- sales_quantity: integer (nullable = true)
 |-- sales_per_product_per_order: double (nullable = true)
 |-- payment_status: string (nullable = true)
```

```
[Stage 120:================================================>      (174 + 6) / 200]
```

```
+-------------+--------+-----------+----------+----------+-------------+---------------------------+-------------
-+
|order_line_id|order_id|customer_id|order_date|product_id|sales_quantity|sales_per_product_per_order|payment_statu
s|
+-------------+--------+-----------+----------+----------+-------------+---------------------------+-------------
-+
|26_1         |26      |145        |2024-06-07|18        |10           |334.5                      |completed
|
|26_2         |26      |145        |2024-06-07|24        |8            |264.32                     |completed
|
|26_3         |26      |145        |2024-06-07|50        |7            |718.76                     |completed
|
|26_4         |26      |145        |2024-06-07|54        |10           |1591.3                     |completed
|
|29_1         |29      |501        |2023-06-21|4         |4            |335.12                     |failed
|
+-------------+--------+-----------+----------+----------+-------------+---------------------------+-------------
-+
```

Note: The data type of **order_date** column is **string**.

- Change data type of order_date to **date**:

```
orders_final = orders_final.withColumn("order_date", to_date(col("order_date"), "yyyy-MM-dd"))

orders_final.printSchema()
```

```
root
 |-- order_line_id: string (nullable = true)
 |-- order_id: long (nullable = true)
 |-- customer_id: long (nullable = true)
 |-- order_date: date (nullable = true)
 |-- product_id: long (nullable = true)
 |-- sales_quantity: integer (nullable = true)
 |-- sales_per_product_per_order: double (nullable = true)
 |-- payment_status: string (nullable = true)
```

- Create the fact_sales table and add the **"order_item_sk"** (surrogate key):

```
windowSpec = Window.orderBy("order_line_id")

fact_sales = orders_final.withColumn(
    "order_item_sk", row_number().over(windowSpec)
).select(
    "order_item_sk",
    "order_line_id",
    "order_id",
    "customer_id",
    "order_date",
    "product_id",
    "sales_quantity",
    col("sales_per_product_per_order").alias("sales_amount"),
    "payment_status"
)
```

- Show the shape of fact_sales table:

```
fact_sales.show(5)
```

```
2026-09-05 01:53:53,811 WARN window.WindowExec: No Partition Defined for Window operation! Moving all data to a si
ngle partition, this can cause serious performance degradation.
[Stage 147:=====================================>              (145 + 6) / 200]
```

```
+-------------+-------------+--------+-----------+----------+----------+--------------+------------+--------------
+
|order_item_sk|order_line_id|order_id|customer_id|order_date|product_id|sales_quantity|sales_amount|payment_statu
s|
+-------------+-------------+--------+-----------+----------+----------+--------------+------------+--------------
+
|            1|        100_1|     100|        434|2024-01-21|        28|             8|      226.08|        faile
d|
|            2|        100_2|     100|        434|2024-01-21|        39|             9|      418.59|        faile
d|
|            3|        100_3|     100|        434|2024-01-21|        59|             5|      383.05|        faile
d|
|            4|        101_1|     101|        575|2024-10-24|         9|             6|        74.4|     complete
d|
|            5|        101_2|     101|        575|2024-10-24|        14|             8|       262.4|     complete
d|
+-------------+-------------+--------+-----------+----------+----------+--------------+------------+--------------
+
only showing top 5 rows
```

- Create the dim_date table:

```
min_date = fact_sales.select(min("order_date")).first()[0]
max_date = fact_sales.select(max("order_date")).first()[0]

dim_date = (
    spark.range(1)
    .select(
        explode(
            sequence(
                lit(min_date),
                lit(max_date),
                expr("interval 1 day")
            )
        ).alias("order_date")
    )
    .select(
        col("order_date").alias("full_date"),
        dayofmonth(col("order_date")).alias("day_of_month"),
        month(col("order_date")).alias("month_number"),
        date_format(col("order_date"), "MMMM").alias("month_name"),
        concat(lit("Q"), quarter(col("order_date"))).alias("quarter"),
        year(col("order_date")).alias("year")
    )
    .orderBy("full_date")
)
```

- Print the Schema and the shape of the dim_date table:

```
dim_date.printSchema()

dim_date.show(5)
```

```
root
 |-- full_date: date (nullable = false)
 |-- day_of_month: integer (nullable = false)
 |-- month_number: integer (nullable = false)
 |-- month_name: string (nullable = false)
 |-- quarter: string (nullable = false)
 |-- year: integer (nullable = false)

+----------+------------+------------+----------+-------+----+
| full_date|day_of_month|month_number|month_name|quarter|year|
+----------+------------+------------+----------+-------+----+
|2023-01-04|           4|           1|   January|     Q1|2023|
|2023-01-05|           5|           1|   January|     Q1|2023|
|2023-01-06|           6|           1|   January|     Q1|2023|
|2023-01-07|           7|           1|   January|     Q1|2023|
|2023-01-08|           8|           1|   January|     Q1|2023|
+----------+------------+------------+----------+-------+----+
only showing top 5 rows
```

- Save the fact_sales table and dim_date table as parquet files:

```
fact_sales.coalesce(1).write.mode("overwrite").parquet("/user/student/e-commerce/cleaned_data/orders")
```

```
2026-09-05 01:53:59,473 WARN window.WindowExec: No Partition Defined for Window operation! Moving all data to a si
ngle partition, this can cause serious performance degradation.
```

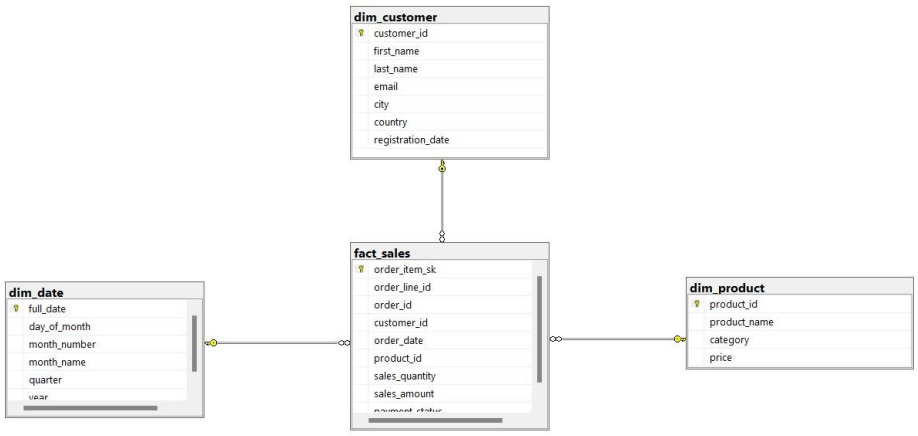
```
dim_date.coalesce(1).write.mode("overwrite").parquet("/user/student/e-commerce/cleaned_data/dates")
```

- List all files, directories and subdirectories inside **"cleaned_data"** directory:

```
[student@localhost ~]$ hdfs dfs -ls -R /user/student/e-commerce/cleaned_data/
drwxr-xr-x   - student student          0 2026-09-05 00:34 /user/student/e-commerce/cleaned_data/customers
-rw-r--r--   1 student student      41315 2026-09-05 00:34 /user/student/e-commerce/cleaned_data/customers/customers.csv
drwxr-xr-x   - student student          0 2026-09-05 01:54 /user/student/e-commerce/cleaned_data/dates
-rw-r--r--   1 student student          0 2026-09-05 01:54 /user/student/e-commerce/cleaned_data/dates/_SUCCESS
-rw-r--r--   1 student student       5123 2026-09-05 01:54 /user/student/e-commerce/cleaned_data/dates/part-00000-0e032d22-6487-4069-969b-f026
b1ab3823-c000.snappy.parquet
drwxr-xr-x   - student student          0 2026-09-05 01:54 /user/student/e-commerce/cleaned_data/orders
-rw-r--r--   1 student student          0 2026-09-05 01:54 /user/student/e-commerce/cleaned_data/orders/_SUCCESS
-rw-r--r--   1 student student      52685 2026-09-05 01:54 /user/student/e-commerce/cleaned_data/orders/part-00000-6dfdfd2a-1e56-4568-ac50-726
8acd63685-c000.snappy.parquet
drwxr-xr-x   - student student          0 2026-09-05 00:35 /user/student/e-commerce/cleaned_data/products
-rw-r--r--   1 student student       2549 2026-09-05 00:35 /user/student/e-commerce/cleaned_data/products/products.csv
[student@localhost ~]$
```

➢ Data Modeling:

We design the **Star Schema**:



Fact Table: "fact_sales"

Dimension Tables: "dim_customer", "dim_product", "dim_date"

➢ **Data Warehouse:**

We use Apache Hive to build a Data Warehouse to make it easy for BI and Data Analysts for reporting:

- Create the Database:

```
CREATE DATABASE IF NOT EXISTS ecommerce;
USE ecommerce;
```

- Create Fact Table:

```
CREATE EXTERNAL TABLE IF NOT EXISTS fact_sales (
    order_item_sk     BIGINT,
    order_line_id     STRING,
    order_id          INT,
    customer_id       INT,
    order_date         DATE,
    product_id        INT,
    sales_quantity    INT,
    sales_amount      DECIMAL(10,2),
    payment_status    STRING
)
STORED AS PARQUET
LOCATION '/user/student/e-commerce/cleaned_data/orders'
TBLPROPERTIES ('parquet.compress'='SNAPPY');
```

- Create Dimension Customer Table:

```
CREATE EXTERNAL TABLE IF NOT EXISTS dim_customer (
    customer_id       INT,
    first_name        STRING,
    last_name         STRING,
    email             STRING,
    city              STRING,
    country           STRING,
    registration_date DATE
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
LOCATION '/user/student/e-commerce/cleaned_data/customers'
TBLPROPERTIES('skip.header.line.count'='1');
```

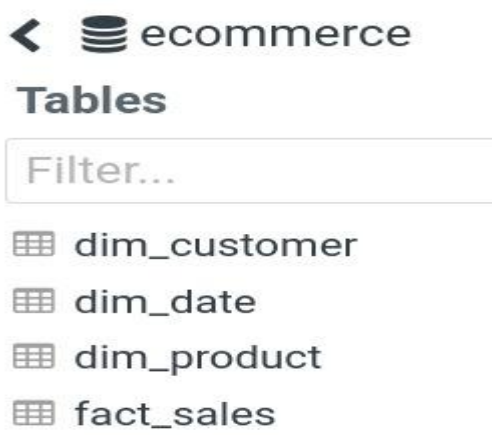
- Create Dimension Product Table:

```
CREATE EXTERNAL TABLE IF NOT EXISTS dim_product (
    product_id        INT,
    product_name      STRING,
    category          STRING,
    price             DOUBLE
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
LOCATION '/user/student/e-commerce/cleaned_data/products'
TBLPROPERTIES('skip.header.line.count'='1');
```

- Create Dimension Date Table:

```
CREATE EXTERNAL TABLE IF NOT EXISTS dim_date (
    full_date         DATE,
    day_of_month      INT,
    month_number      INT,
    month_name        STRING,
    quarter           STRING,
    year              INT
)
STORED AS PARQUET
LOCATION '/user/student/e-commerce/cleaned_data/dates'
TBLPROPERTIES ('parquet.compress'='SNAPPY');
```

The output:



- Explore the **fact_sales** table:

```
SELECT *
FROM ecommerce.fact_sales
LIMIT 10;
```

	fact_sales.order_item_sk	fact_sales.order_line_id	fact_sales.order_id	fact_sales.customer_id	fact_sales.order_date
1	1	100_1	100	434	2024-01-21
2	2	100_2	100	434	2024-01-21
3	3	100_3	100	434	2024-01-21
4	4	101_1	101	575	2024-10-24
5	5	101_2	101	575	2024-10-24
6	6	101_3	101	575	2024-10-24
7	7	101_4	101	575	2024-10-24
8	8	101_5	101	575	2024-10-24
9	9	102_1	102	59	2024-09-06
10	10	102_2	102	59	2024-09-06

fact_sales.product_id	fact_sales.sales_quantity	fact_sales.sales_amount	fact_sales.payment_status
28	8	226.08	failed
39	9	418.59	failed
59	5	383.05	failed
9	6	74.4	completed
14	8	262.4	completed
36	6	387.72	completed
36	5	323.1	completed
37	5	96.95	completed
4	8	670.24	completed
38	5	1101.45	completed

- Explore the **dim_customer** table:

```
5 SELECT *
6 FROM ecommerce.dim_customer
7 LIMIT 10;
```

	dim_customer.customer_id	dim_customer.first_name	dim_customer.last_name	dim_customer.email
1	1	Joseph	Brown	joseph.brown@outlook.com
2	2	Carlos	Moore	carlos.moore@gmail.com
3	3	Marco	Lopez	marco.lopez@hotmail.com
4	4	Patricia	Lopez	patricia.lopez@yahoo.com
5	5	Bjorn	Brown	bjorn.brown@yahoo.com
6	6	Marco	Wang	marco.wang@hotmail.com
7	7	Greta	Smith	greta.smith@proton.me
8	8	Yusuf	Baker	yusuf.baker@yahoo.com
9	9	Amara	Wilson	amara.wilson@hotmail.com
10	10	Ngozi	Hall	ngozi.hall@outlook.com

dim_customer.city	dim_customer.country	dim_customer.registration_date
Kuala Lumpur	Malaysia	2022-09-08
Kuala Lumpur	Malaysia	2023-11-24
Johannesburg	South Africa	2022-02-02
Lyon	France	2023-06-02
Lagos	Nigeria	2023-10-28
Berlin	Germany	2023-08-27
Jakarta	Indonesia	2022-06-13
Barcelona	Spain	2022-08-09
Vancouver	Canada	2022-04-10
Accra	Ghana	2022-02-14

- Explore the **dim_product** table:

```
 8
 9 SELECT *
10 FROM ecommerce.dim_product
11 LIMIT 10;
```

	dim_product.product_id	dim_product.product_name	dim_product.category	dim_product.price
1	1	Non-Stick Frying Pan	Home & Garden	86.40
2	2	Webcam HD	Electronics	236.02
3	3	Jump Rope	Sports	79.23
4	4	Scented Candle Set	Home & Garden	83.78
5	5	Puzzle 1000pc	Toys	35.54
6	6	Yoga Mat	Sports	56.00
7	7	Building Blocks Set	Toys	48.59
8	8	Leather Belt	Clothing	64.70
9	9	Moisturizing Lotion	Beauty	12.40
10	10	Hiking Backpack	Sports	196.36

- Explore the **dim_date** table:

```
12
13 SELECT *
14 FROM ecommerce.dim_date
15 LIMIT 10;
```

	dim_date.full_date	dim_date.day_of_month	dim_date.month_number	dim_date.month_name	dim_date.quarter
1	2023-01-04	4	1	January	Q1
2	2023-01-05	5	1	January	Q1
3	2023-01-06	6	1	January	Q1
4	2023-01-07	7	1	January	Q1
5	2023-01-08	8	1	January	Q1
6	2023-01-09	9	1	January	Q1
7	2023-01-10	10	1	January	Q1
8	2023-01-11	11	1	January	Q1
9	2023-01-12	12	1	January	Q1
10	2023-01-13	13	1	January	Q1

➢ Dashboard & Analytics:

We use Power BI to make dashboard and analyze KPIs:

- Key Insights:

1. Smart Watch is the Top-Selling Product: Out of all listed products, the Smart Watch leads sales, followed closely by the Mechanical Keyboard and Webcam HD.
2. **India drives the Highest Regional Sales**: India is the top country by sales revenue, outperforming other key markets like the USA and China.
3. **Payment Failure and Refund Rates Need Attention:** Nearly half of all transactions fail or end up refunded (233 failed, 227 refunded), which indicates a potential issue with the payment gateway or user checkout process.